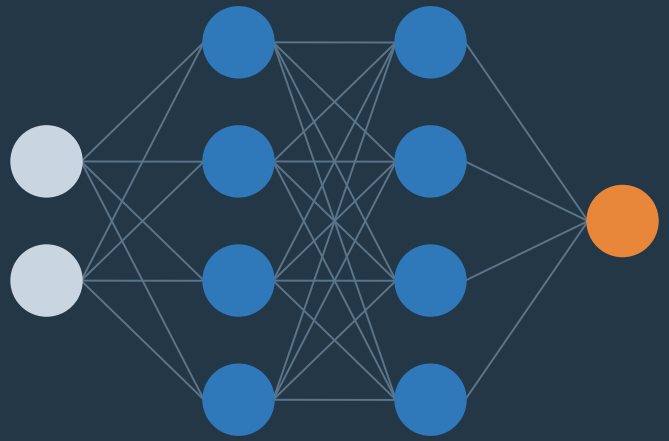
Inside a Neural Network

How layers, neurons, weights, activation functions, loss functions, and optimizers work together



Shailu Beesupaka · AIML-501 Model Development · Indiana Wesleyan University

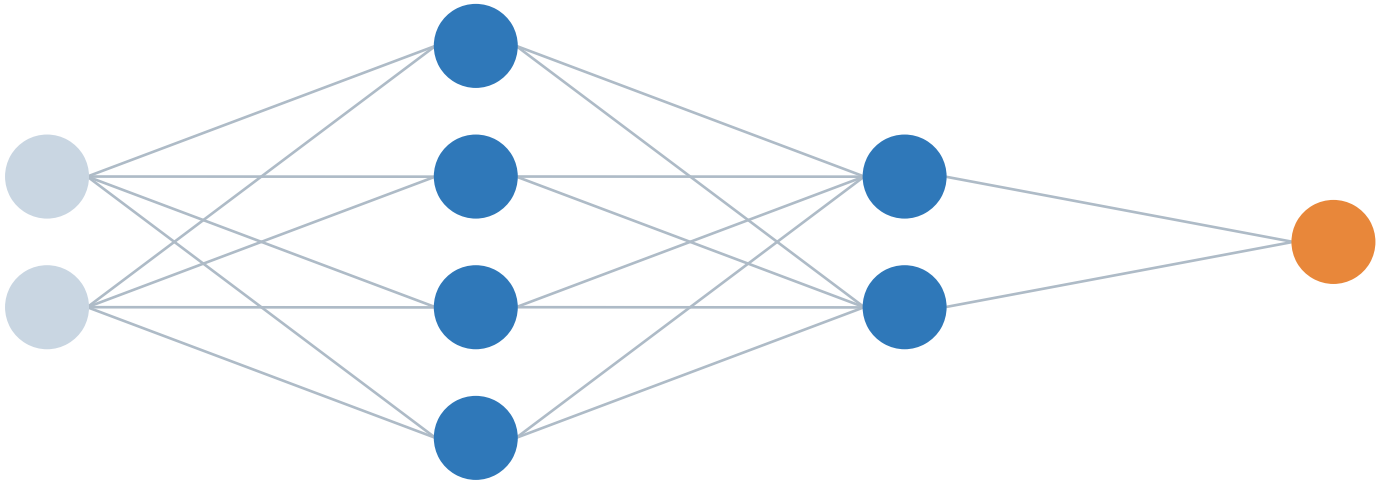
How data flows through a network

Input layer
features X1, X2

Hidden layer 1
4 neurons

Hidden layer 2
2 neurons

Output layer
1 prediction



Forward pass: inputs move left to right and become a prediction

Backward pass (training only): the error flows back to adjust every weight

Shown: the Playground's default network. Learning type: supervised. Task: binary classification (blue = +1, orange = −1). Sources: Goodfellow et al. (2016); Smilkov et al. (2017)

1 Layers organize the work

Input layer

Receives the raw features. In the Playground, these are each point's X1 and X2 coordinates. It does no calculation.



Hidden layers

Transform the data step by step. Each layer builds more complex patterns from the simpler patterns found by the layer before.



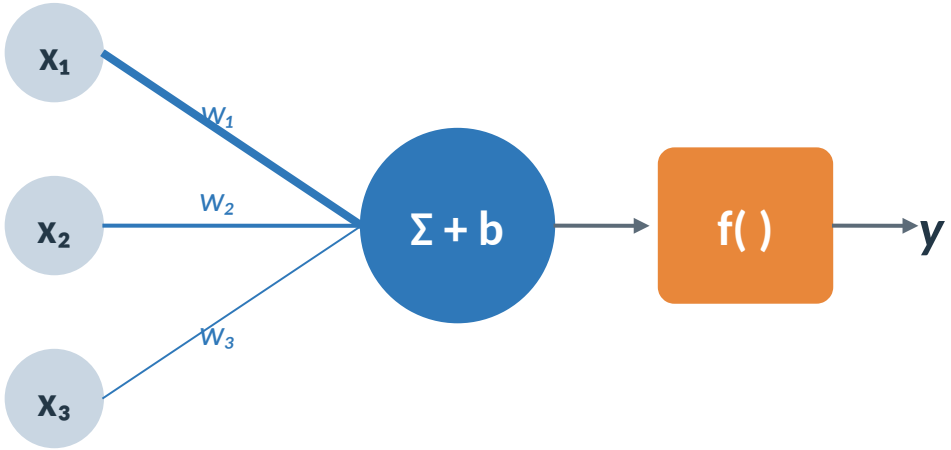
Output layer

In the Playground, one tanh neuron outputs a value from −1 to 1. Positive values mean blue (+1); negative values mean orange (−1).

Deep learning simply means a network with many hidden layers. My third Playground experiment used four.

Sources: Goodfellow et al. (2016); Google for Developers (n.d.); TensorFlow (n.d.-a)

2 A neuron is a small calculator



$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

1. **Weigh**
   Multiply each input by its weight.
2. **Add**
   Sum the results and add a bias (b).
3. **Activate**
   Pass the total through an activation function f to get the output y.
4. **Pass on**
   In a fully connected network like the Playground's, y goes to every neuron in the next layer.

Sources: Goodfellow et al. (2016); Google for Developers (n.d.)

3 Weights decide what matters

25

learnable values in the Playground's default network: 18 weights + 7 biases

151

in my 4-layer network (126 weights + 25 biases)



Strength
A larger weight gives an input more influence on the neuron's output.

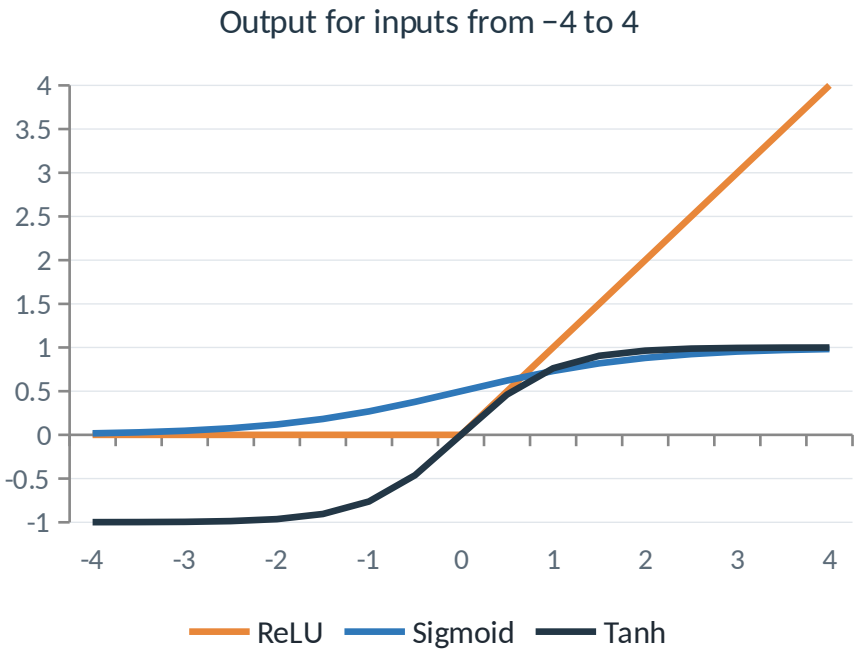
Direction
Positive weights keep an input's effect; negative weights reverse it. In the Playground: blue = positive, orange = negative, thicker = larger.

Learned, not set
In the Playground, weights start as random numbers between −0.5 and 0.5. Training then adjusts them to reduce the loss.

Bias
Each hidden and output neuron also has a bias (starting at 0.1 in the Playground) that shifts when the neuron becomes active.

Counts: weights = connections between layers; biases = one per hidden and output neuron. Sources: Goodfellow et al. (2016); TensorFlow (n.d.-a)

4 Activation functions let networks bend



ReLU *max(0, x)*
Passes positives, blocks negatives. Fast and the most common choice for hidden layers.

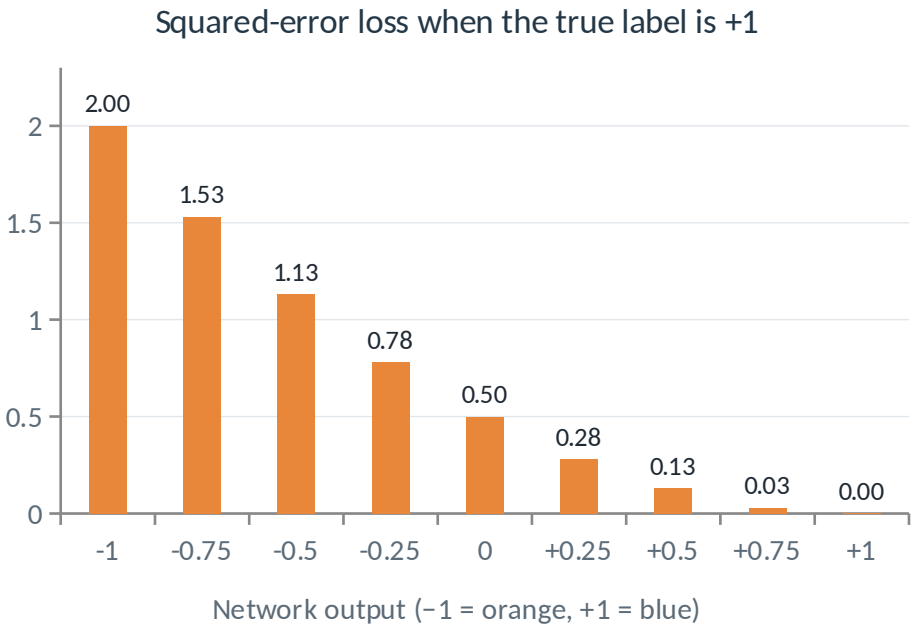
Sigmoid *squashes to 0 to 1*
Useful for outputs that are probabilities.

Tanh *squashes to −1 to 1*
Centered on zero. The Playground's default.

Without them, any stack of layers collapses into a single linear model, which can only draw a straight boundary, never a circle or a spiral.

Sources: Goodfellow et al. (2016); Nair & Hinton (2010)

5 Loss functions measure mistakes



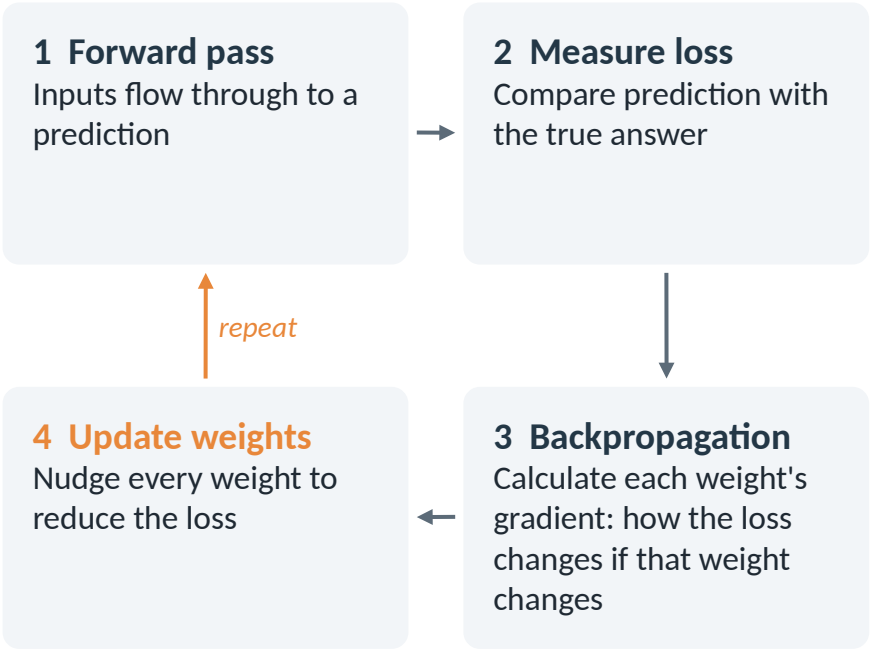
Squared error (used by the Playground)
Loss = ½ × (output − label)² for each point, averaged over all points. A perfect prediction scores 0; the worst possible scores 2.

Cross-entropy (common elsewhere)
Many classification models use cross-entropy instead, which punishes confident wrong answers even more heavily.

Every loss value in my experiments is this average squared error, measured on the 250 test points.

Chart values calculated as 0.5 × (output − 1)². Sources: Goodfellow et al. (2016); TensorFlow (n.d.-a)

6 Optimizers turn mistakes into learning



new weight = old weight – learning rate × gradient

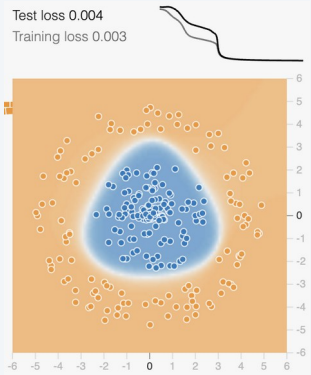
Mini-batch gradient descent
Averages gradients over a small batch, then updates. The Playground updates after every 10 points.

Adam
Adapts each weight's step size automatically. Widely used, but not in the Playground.

Learning rate
Too high overshoots the best weights; too low learns slowly. The Playground default is 0.03.

Sources: Rumelhart et al. (1986); Kingma & Ba (2015); TensorFlow (n.d.-a)

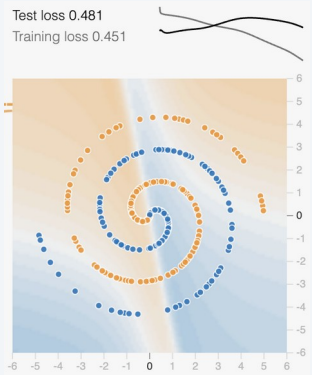
My TensorFlow Playground experiments



1 · Circle data

2 hidden layers (4, 2) · Tanh
323 epochs

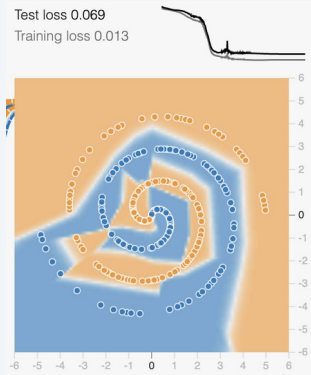
0.004 test loss



2 · Spiral, same network

2 hidden layers (4, 2) · Tanh
313 epochs

0.481 test loss



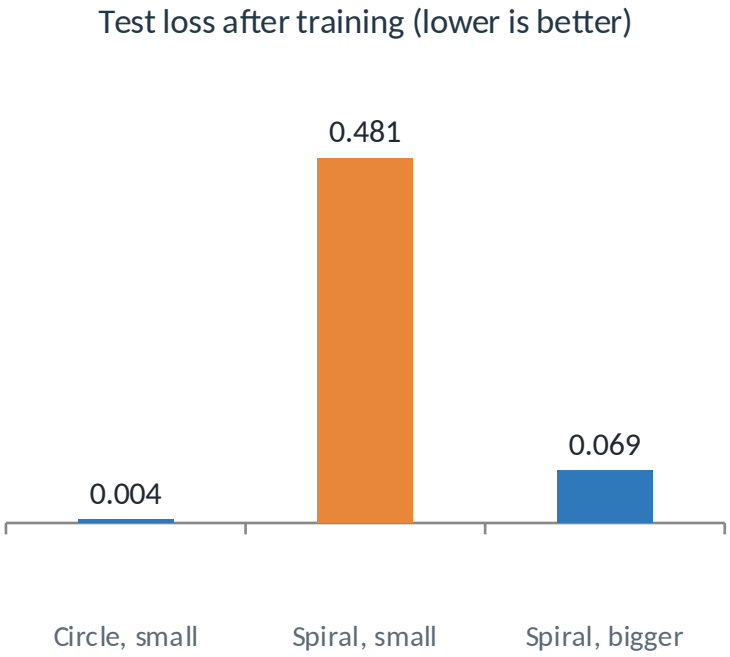
3 · Spiral, bigger network

4 hidden layers (6 each) · ReLU
512 epochs

0.069 test loss

All runs: supervised binary classification, 500 points (250 train, 250 test), learning rate 0.03, batch 10, noise 0, no regularization, inputs X1, X2. TensorFlow (n.d.-b)

What the experiments showed me



The data sets the difficulty. After about 300 epochs, the same small network reached 0.004 on the circle but only 0.481 on the spiral.

More layers and ReLU solved the spiral. Test loss dropped about 86%, from 0.481 to 0.069, and the boundary followed the spiral's arms.

Bigger has a cost. Learnable values grew from 25 to 151, and training loss (0.013) ended below test loss (0.069), a gap that can signal overfitting.

You can see ReLU at work. The spiral's boundary is made of straight-edged pieces, because networks of ReLU neurons build piecewise-linear boundaries.

Values from my three Playground runs. 86% = (0.481 − 0.069) ÷ 0.481. Starting weights are random, so repeat runs vary slightly.

Why visualizing neural networks matters

Why it matters

- It opens the black box: you can watch weights and decision boundaries change as the network learns.
- It makes problems visible, such as a network too small for its data or one starting to overfit.
- It helps explain AI to non-technical audiences without a single equation.

Key insights I gained

- Each component has one job, and together they form a loop: predict, measure the loss, backpropagate, update.
- Activation functions are what let networks learn curved patterns.
- The right network size depends on the data, and the gap between training and test loss is the warning light.

Watching the network learn turned six separate definitions into one connected system.

References

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org/

Google for Developers. (n.d.). *Neural networks*. Machine Learning Crash Course. https://developers.google.com/machine-learning/crash-course/neural-networks

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.1412.6980

Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning* (pp. 807–814).

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536. https://doi.org/10.1038/323533a0

Smilkov, D., Carter, S., Sculley, D., Viégas, F. B., & Wattenberg, M. (2017). *Direct-manipulation visualization of deep networks* (arXiv:1708.03788). arXiv. https://doi.org/10.48550/arXiv.1708.03788

TensorFlow. (n.d.-a). *Playground* [Source code]. GitHub. https://github.com/tensorflow/playground

TensorFlow. (n.d.-b). *A neural network playground* [Online tool]. https://playground.tensorflow.org/